# ERP System Project - TODO List

Based on the core project requirements, architecture guidelines in `README.md`, and recent updates, this list tracks the necessary tasks. **This file should be continuously updated as the project progresses.**

## 🔐 Phase 1: Authentication & Authorization (Backend First)

Implement secure access and user management focusing on Google Account integration.

- [x] **Google OAuth Setup**: Configure Google Cloud Console credentials for OAuth2.
- [x] **Backend Auth API**: Implement `/api/auth/google` (login/signup) and `/api/auth/logout` endpoints.
- [x] **Token Management**: Implement JWT (JSON Web Token) generation and management upon successful Google authentication.
- [x] **Global Token Validation Middleware**: Create a robust filter/middleware in `src/backend/common/middleware/authMiddleware.js` to intercept ALL API calls and validate the issued token.
- [x] **User Model Update**: Ensure `src/backend/module_sys/models/user.js` can handle Google OAuth identities (e.g., Google ID, email).

## 📊 Phase 2: API Logging System

Ensure all interactions with the backend are recorded for auditing and debugging.

- [ ] **API Logging Middleware**: Implement middleware in `src/backend/common/config/logger.js` (or a dedicated middleware) to log every incoming API request (Method, URL, User ID from Token, Timestamp, Response Status).
- [ ] **Database/File Logging**: Configure the logger to write to a persistent store (database table or rotating log files).

## 🏗 Phase 3: Backend Service Architecture (API-First)

Ensure the backend acts as a fully capable, standalone service.

- [ ] **API Completeness**: Ensure all business logic across `module_sys` is fully executable via RESTful API calls without depending on frontend state.
- [ ] **Standardized API Responses**: Implement consistent success/error response formatting across all controllers.
- [x] **Layer Separation (Service/Repository)**: Separate DB queries into Repository files (e.g., `roleRepository.js`) to keep Services focused on business logic.

## 💻 Phase 4: Lightweight Frontend Integration

Keep the frontend thin, delegating heavy logic to the backend APIs.

- [x] **Google Login/Signup UI**: Implement a simple login page (`AuthLayout.html` / `Login.vue`) with a "Sign in with Google" button.

- ☑ **Token Storage**: Securely store the JWT on the client side (e.g., HttpOnly cookies or secure localStorage).
- ☑ **Axios/Fetch Interceptors**: Configure frontend API clients to automatically attach the token to all outgoing requests.
- ☑ **Logout Functionality**: Implement frontend 기state clearing and backend logout API call.
- ☑ **API Modularization**: Move inline `fetch` calls from Vue components into dedicated service modules (e.g., `src/frontend/module_sys/api/menuApi.js`) using Axios.

## 🎨 Phase 5: Implement Missing Common UI Components (From `README.md`)

- ☑ Create `AppHeader.vue` (Layout component for top bars)
- ☑ Create `AppFooter.vue` (Layout component for bottom bars)
- ☑ Create `AppPageTitle.vue` (Standard page title component)
- ☑ Create `AppGrid.vue` (Flexbox-based grid system)
- ☑ Create `AppTable.vue` (Standardized data table component)
- ☑ Create `AppModal.vue` (Modal dialog component)

## ⚙️ Phase 6: System Module (SYS) UI Implementation & Refactoring

Implement and refactor the frontend screens corresponding to the core system models, adhering to the new folder and naming conventions (List/Detail separation).

- ☑ **SYST01 User Management**: Implement `SYST01V001.vue` (List) and `SYST01V002.vue` (Detail/Form).
- ☑ **SYST02 Role & Permission**: Implement `SYST02V001.vue` (List) and `SYST02V002.vue` (Detail/Form).
- ☑ **SYST03 Menu Management**: Implement `SYST03V001.vue` (List) and `SYST03V002.vue` (Detail/Form).
- ☑ **SYST04 Common Code**: Implement `SYST04V001.vue` (List) and `SYST04V002.vue` (Detail/Form).
- ☑ **SYST05 System Settings**: Implement `SYST05V001.vue` (List) and `SYST05V002.vue` (Detail/Form).
- ☑ **Bugfix: SYST05 System Settings (`configService.js`)**:
  - ☑ Fix "Config not found" error when updating mock data. (Seed initial data to DB)
  - ☑ Fix issue where mock data disappears after a new config is created.
  - ☑ Allow updates without changes (handle `updatedRows === 0` gracefully).

## ⚙️ Phase 8: SYST06 Table Specifications Manager Implementation

Implement the module to manage database table specifications, track change history, and generate SQL DDL scripts.

- ☑ **Backend: Table Spec Models**: Verify/Create Sequelize or Raw SQL models for `sysTable`, `sysTablex`, `sysFields`, `sysFieldsx`, `sysTableIndex`, `sysTableIndexx`, `sysIndexFields`, `sysIndexFieldsx`, and `sysTableHistory`.
- ☑ **Backend: CRUD API**: Create `tableSpecController.js` and `tableSpecRoutes.js` for fetching, creating, and updating table specs.
- ☑ **Backend: History & SQL Generation Service**: Create `tableSpecService.js` to automatically detect changes, log to `sysTableHistory`, and generate DDL (`CREATE TABLE`, `ALTER TABLE`, `CREATE INDEX`).

- ☑ **Frontend: Views Creation**: Create `SYST06/SYST06V001.vue` (List View) and `SYST06/SYST06V002.vue` (Detail View).
- ☑ **Frontend: Master-Detail UI**: Implement Master form (Table info) and Detail tabs/grids (Fields, Indexes, History) in `SYST06V002`.
- ☑ **Frontend: SQL Execution UI**: Add "Generate SQL" modal and execution functionality to apply changes to the DB.
- ☑ **Bugfix & UX/UI: SYST06**:
  - ☑ Fix Cartesian Product (N+1) bug in `tableSpecService` causing slow loads and duplicated index IDs.
  - ☑ Add up/down reorder buttons to Fields and Indexes lists.
  - ☑ Support AUTO_INCREMENT configuration in Fields (`isAutoIncrement`) and DDL generation.
  - ☑ Fix Vue Router path mismatch (`/sys/tables` to `/sys/syst06`).
  - ☑ **Modern UI Refactoring**: Completely refactored `SYST05V002` and `SYST06V002` to use responsive flex/grids, staggered CSS animations (`fadeInUp`), and modern input/button styles (`icon-btn`, `btn-danger`, transparent table cells).
- ☑ **SYST00 System Dashboard**: Implement `SYST00V001.vue` (Dashboard).

## 🔍 Phase 7: Mandatory Search, Pagination & UI Modernization Implementation

Ensure all list views contain proper search conditions, a search button, pagination, and adhere strictly to the new modern UX/UI guidelines.

- ☑ **UI Component Update**: Create or update existing pagination and search bar UI components.
- ☑ **System Module (SYS) Update**: Apply search filters, pagination, and the new **modern UI classes** (smooth animations, responsive auto-fit grids, modern-data-table) to the list views: `SYST01V001`, `SYST02V001`, and `SYST04V001`.
- ☐ **Strict API Modularization**: Refactor any remaining views (SYST02, SYST03, SYST05, SYST06, SYST00) to strictly use API modules (`api/*Api.js`) and completely remove raw `fetch`/`axios` calls.
- ☑ **Strict API Modularization**: Refactor any remaining views (SYST02, SYST03, SYST05, SYST06, SYST00) to strictly use API modules (`api/*Api.js`) and completely remove raw `fetch`/`axios` calls.
- ☐ **Global Standard Enforced**: Verify that any future view screens adhere to this mandatory modern standard.

## 🔧 Ongoing Maintenance

- ☐ **System Variables Documentation**: Maintain documentation of all required `.env` variables (e.g., `DB_HOST`, `VITE_GOOGLE_CLIENT_ID`, `JWT_SECRET`) in `README.md`.
- ☐ **Global Constants & Formatters**: Update `constants.js` and `formatters.js` files as new system-wide logic, magic strings, or numbers are discovered.
- ☐ **Update Documentation**: Continuously update `README.md` with new API endpoints, middleware explanations, and auth flow diagrams.
- ☐ **Update Logs**: Record significant architecture changes and completions in `gemini.log`.
- ☐ **Update TODO.md**: Check off completed tasks and add new ones as discovered. discovered.